

FAI Conflict Detection at Merge

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 15, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the inter-Self coordination architecture introduced in “Inter-Self Coordination via Shared Substrate / Full Aspect Integration” (Li, April 2026), the third paper in the CKS theory series following “Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems” (Li, April 2026) and “The Instinct/Reasoning Separation Outside the Model” (Li, April 2026).

Abstract

D2.07 is the operational decomposition of D1.08, which committed that full-merge produces conflicts registered as first-class objects in the shared substrate. This note formalizes what that commitment means at the operational level: how conflicts are detected as contributed aspects are merged, what each conflict record contains, how detected conflicts are routed to the appropriate tier of the three-tier handling mechanism, and what aggregate conflict density communicates to governance. Three architectural properties receive precise treatment. First, conflict detection is integral to the merge operation—it is not a post-hoc review phase; detection and merge are the same pass. Second, the routing rules that assign detected conflicts to tiers are themselves authored orchestration rules in the shared substrate—routing is governed, not hard-coded. Third, conflict density is a governance compatibility signal rather than a failure indicator—high density communicates that contributing Selves’ aspects approach overlapping operational territory differently, which is information governance needs for home evolution decisions. The note states the anti-pattern of silent conflict collapse and provides an operational test an observer can apply to verify that a merge event produced compliant conflict records.

1. D2.07 as Operational Decomposition of D1.08

D1.08 committed that full-merge is the architectural default for FAI events—contributed aspects from participating Selves are merged in the shared substrate in full, and where that merge produces conflicts, the conflicts are registered as first-class objects. That commitment answers a design question about what the merge default is. It does not answer the operational question: when contributing aspects are merged, what procedures produce the first-class conflict objects, and what happens to those objects afterward?

D2.07 answers that question. It is not an independent architectural commitment; it is the operational specification of D1.08's output. If D1.08 is the commitment that first-class conflict objects exist, D2.07 is the specification of how they come to exist, what they contain, and what the shared substrate does with them next.

The operational decomposition matters for defensive publication because the patent surface of conflict handling is not located primarily in the commitment to treat conflicts as first-class objects. It is located in the mechanism that detects them, the structure of the records that carry them, the governed routing that assigns them to tiers, and the governance interpretation of the aggregate detection signal. Each of these is a formalizable commitment, and each is derived from the source papers' architectural commitments rather than introduced fresh here.

2. What Constitutes a Conflict at Inter-Self Merge

The shared substrate holds contributed aspects from two or more participating Selves. Each aspect is a purpose-defined arrangement of cells, carrying DNA-layer and action-layer content from the contributing Self's home governance. When these aspects are merged, a conflict arises wherever two or more contributed aspects have overlapping operational territory with divergent specifications.

Three forms of overlap produce detectable conflicts:

Input-class overlap with divergent response specifications. Two aspects both carry rules governing the same class of inputs, but specify different responses. When merged, the shared substrate cannot apply both rules without contradiction for inputs in that class.

Content-domain overlap with ambiguous governance. Two aspects carry content-domain specifications—descriptions of what subject matter the aspect governs—that overlap, creating ambiguity about which aspect's rules are authoritative for inputs that fall in the intersection.

Orchestration-rule contradiction. Two aspects carry orchestration rules that, when applied to the same substrate state, produce contradictory outputs. The contradiction need not involve identical input classes; it may arise from rules that are each consistent in isolation but produce divergent outputs when the substrate state is the product of both aspects being present.

What does not constitute a conflict, under this specification: non-overlapping aspects that cover different operational territory without contradiction. The existence of multiple contributed aspects is not itself a conflict. The conflict criterion requires both overlap and divergence. Aspects covering different domains without intersection may coexist in the shared substrate without producing conflict records.

3. Detection Is Integral to the Merge Operation

Conflict detection is not a review phase that follows the merge operation. It is part of the merge operation itself. The detection pass—comparing contributed aspects’ content-domains, orchestration rules, and input-class specifications for overlap and divergence—occurs as aspects are merged in the shared substrate. Detection and merge are the same pass.

This design decision is architecturally load-bearing. If detection were post-hoc—if the merge completed first, and then a separate process reviewed the merged result for conflicts—the comprehensiveness of conflict registration would depend on the scope and scheduling of that separate process. Missed conflicts would produce a substrate that silently contains contradictions, violating Paper 1 Claim 2’s conflict-preservation commitment at inter-Self scope.

When detection is integral to merge, the constraint is different: the merge operation cannot complete without having run detection on the aspects being merged. Every aspect pair that enters the merge is compared; every overlap-with-divergence that comparison finds produces a conflict record before the merge step that introduces the overlapping content advances. You cannot merge without detecting. This is what makes conflict registration comprehensive at inter-Self scope.

The practical implication: an implementation that completes a merge operation without producing any conflict records is asserting either that no overlap-with-divergence was found, or that detection was skipped. An observer verifying compliance should be able to distinguish between these—the conflict registry should reflect the detection pass, not simply the output of a silent auto-resolution.

4. Conflict Record Contents

Each detected conflict is registered as a first-class object in the shared substrate’s conflict registry, inheriting the conflict-preservation commitment from Paper 1 Claim 2. The conflict record contains four elements.

Both sides preserved. Both conflicting specifications are retained in the conflict record as substrate content. Neither is discarded at detection time. The merge operation places both sides of the conflict into the registry rather than selecting one, averaging them, or substituting a

default. This is the operational expression of “conflict preservation as architectural default”: the default output of detecting a conflict is that both sides exist as addressable substrate state.

Attribution with full provenance. The conflict record identifies which contributing Self’s aspect produced each side of the conflict. Attribution is not an optional annotation; it is a required field in the record. The provenance chain connects each side to the contributing aspect, and through the aspect to the contributing Self’s home governance. This allows governance—both the shared substrate’s joint governance and each participating Self’s home governance—to trace a detected conflict back to its origins during any subsequent review.

Conflict class. If the conflict matches a known class for which the shared substrate carries pre-authored orchestration rules, the record notes the class identifier. Conflict classes are defined by governance as part of the shared substrate’s initial configuration (D2.01): governance authors the rule covering a conflict class, and by doing so implicitly defines what membership in that class means. If the detected conflict does not match any known class, the record notes the absence of a class match. The class field is what routing uses: class-matched conflicts are eligible for the resolve tier; class-unmatched conflicts that require resolution are candidates for escalation.

Detection timestamp. The record carries the time at which the conflict was detected during the merge operation. The timestamp supports path retraceability—an observer following the conflict’s history can establish when in the shared substrate’s operational timeline the conflict entered the registry—and supports governance review of whether routing and tier handling were applied within expected timeframes.

5. Three-Tier Routing After Detection, and Its Governance Character

Each registered conflict is routed to one of three tiers. The routing decision is the first operation applied to a conflict record after detection; it determines what the shared substrate does with the conflict next.

Preserve. If the task does not require that the conflict be resolved during the current FAI event—if the conflict is in territory where both specifications can coexist as substrate content without the event’s purpose requiring a determination between them—the conflict remains in the registry as first-class substrate state. No resolution is attempted. The conflict may be carried through dissolution into each participating Self’s home substrate as an evolution-feed annotation, where it can inform home governance decisions about aspect evolution.

Resolve via pre-authored orchestration rules. If the conflict matches a known class for which pre-authored orchestration rules exist in the shared substrate, those rules determine the response. The rules are substrate content authored by governance as part of the shared

substrate's configuration; executing them is the substrate mediator's role, not a governance authority's live judgment. The resolution is an operation governed by authored rules, not an ad hoc determination.

Escalate to participating Selves' governance. If no orchestration rule covers the conflict class and resolution is needed for the event's purpose, the conflict is surfaced to the joint-governance arrangement of the participating Selves' home governance structures. Escalation is the tier that carries the cross-perimeter joint-authority shape: the conflict cannot be resolved by any single Self's governance unilaterally, because both sides of the conflict carry aspects contributed by different Selves under different home governance perimeters.

Routing is itself governed. The routing rules—the authored logic that determines which tier a given conflict is assigned to based on its class, the current event's purpose, and the shared substrate's configuration—are themselves authored orchestration rules in the shared substrate. They are not hard-coded behavior in the merge implementation; they are substrate content authored by governance as part of initial shared-substrate configuration (D2.01). This means routing decisions are inspectable, modifiable, and overridable under the three governance rights Paper 1 Claim 1 establishes. A governance authority can change the routing logic by modifying the routing rules in the substrate; the change takes effect as authored substrate content. The recursive applicability of substrate-content governance extends to the routing mechanism itself.

The implication for defensive publication: an implementation that routes conflicts to tiers through logic that is not substrate content—through hard-coded behavior, vendor-determined defaults, or runtime logic outside the shared substrate's authored rules—does not instantiate this commitment. The routing mechanism's substrate-content character is the specific formalization D2.07 adds beyond what D1.08 established.

6. Conflict Density as a Governance Compatibility Signal

The merge operation produces not only individual conflict records but an aggregate detection result: the number of conflicts detected across the full merge of all contributed aspects. This aggregate—conflict density—is a governance signal about the compatibility of the contributing Selves' aspects at the moment of the FAI event.

High conflict density means that the contributing Selves' aspects cover significantly overlapping operational territory with significantly different approaches. It does not mean the FAI event failed. It means that the contributing Selves have developed their aspects—under their respective home governance perimeters—in ways that diverge substantially over shared territory. That is information.

The information is useful at two levels. At the shared substrate level, high density means a larger proportion of the merge's output enters the conflict registry rather than coexisting without contradiction. Governance configured the three-tier routing to handle this; the density itself does not change the mechanism, though it may change the load on each tier.

At the home governance level, conflict density from a FAI event feeds into each Self's home evolution machinery (D1.27). A Self whose contributed aspects generated many conflicts against another Self's aspects has learned something about the gap between its current aspect specifications and those of the Selves it coordinates with. That information can inform the Self's governance decisions about how to evolve its aspects—whether to narrow content-domain specifications, revise orchestration rules, or deliberately maintain differentiation as a reflection of genuine governance preference. The decision is the home governance's to make; the conflict density from the FAI event is the evidence it makes it from.

Conflict density is therefore neither a performance metric to minimize nor an error rate to explain away. It is a measurement of the governance distance between contributing Selves' aspects at the time of the merge event, surfaced as substrate content by the detection mechanism and available to governance as evidence for evolution decisions.

7. Inheritance from Paper 1 Claim 2 and Paper 2 Mating Conflict Registration

The conflict detection and registration mechanism specified in D2.07 is not fresh architecture. It inherits from two prior commitments in the trilogy.

Paper 1 Claim 2 commits to conflict preservation as the architectural default at cell scope within a single Self's substrate: conflicts are preserved as substrate content, both sides retained, not auto-resolved. The conflict registry at cell scope is the origin of the pattern. D2.07 applies the same pattern at inter-Self scope: the shared substrate's conflict registry is the cell-scope registry extended to the inter-Self perimeter.

Paper 2's mating conflict registration (B1.12) established the same detection-and-registration mechanism at intra-Self scope: when aspects are combined within a Self through the mating operation, conflicts detected during that combination are registered as first-class objects. The detection-is-integral-to-merge commitment, the both-sides-preserved record structure, and the attribution requirement all appear at intra-Self scope in Paper 2. D2.07 applies them at inter-Self scope.

What is fresh at inter-Self scope is the attribution dimension: the conflict record must now identify which contributing Self's aspect—not just which aspect—produced each side. This is because the cross-perimeter character of the conflict gives attribution a governance dimension it does not have within a single Self's home substrate. When a conflict is between aspects from

two different Selves under two different home governance perimeters, provenance determines which governance authority is relevant for escalation decisions.

8. Anti-Pattern: Silent Conflict Collapse

The named anti-pattern for D2.07 is **silent conflict collapse**: conflicts detected during merge that are auto-resolved—with one side discarded—without first-class registration, before the conflict record is created.

Silent conflict collapse violates Paper 1 Claim 2 at inter-Self scope. The violation occurs at the detection step: a conflict is found, but instead of creating a record with both sides preserved, the detection mechanism immediately selects one side and discards the other, leaving no trace that a conflict existed. The shared substrate’s post-merge state appears consistent because the conflict was erased before it could be registered.

The practical harm of silent conflict collapse is not always visible at merge time. It becomes visible when governance attempts to review what happened during a FAI event, when a participating Self’s home governance wants to understand why its contributed aspect’s specifications were not reflected in the event’s outputs, or when path retraceability requires tracing a substrate state back through the decisions that produced it. If conflicts were silently collapsed, the path is broken: there is no record of the discarded specification, no attribution of which Self’s aspect was overridden, and no basis for the escalation or home evolution decisions that the preserved conflict would have enabled.

An implementation may include convenience mechanisms that auto-resolve common conflict classes—this is the resolve tier operating through pre-authored orchestration rules. That is not silent conflict collapse. The distinction is registration: in compliant auto-resolution, the conflict record is created first (with both sides and attribution), then the resolution rule is applied, and the resolution is recorded in the registry as the outcome of the rule’s execution. In silent conflict collapse, the conflict record is never created. The distinguishing test is whether the conflict registry carries a record showing both sides before resolution logic was applied.

9. Operational Test

For a merge event with conflicts, a compliant implementation satisfies all of the following when examined by an observer with access to the shared substrate’s conflict registry:

1. **Conflict records exist.** The conflict registry contains at least one record for each input-class overlap with divergent specifications, content-domain overlap with ambiguous governance, and orchestration-rule contradiction detected during the merge of contributed aspects.

2. Both sides are present in each record. Each conflict record contains both conflicting specifications as substrate content. Neither has been discarded, overwritten, or replaced with a default at the time of registration.

3. Attribution is complete. Each conflict record identifies the contributing Self (or Selves) whose aspect produced each side of the conflict, with provenance connecting each side to its source aspect.

4. Routing records are present. Each conflict record carries a routing designation—preserve, resolve via orchestration, or escalate—determined by the authored routing rules in the shared substrate. The routing decision is traceable to an authored rule in the substrate, not to hard-coded implementation behavior.

5. No unregistered resolutions exist. There are no substrate states that differ from what full merge without any resolution would produce except where the difference is traceable through the conflict registry. A substrate state that reflects one side of a detected conflict winning over the other, with no conflict record showing that conflict, indicates silent conflict collapse.

6. Conflict density is readable. The conflict registry supports aggregate queries: an observer can determine the total number of conflicts detected during a given merge event, broken down by conflict class if classes are assigned, and by contributing Self pair if attribution is present.

An implementation that satisfies (1)–(6) instantiates the conflict detection and registration commitment D2.07 formalizes. An implementation that fails any of (1)–(5) may produce useful coordination outputs, but does not satisfy Paper 1 Claim 2 at inter-Self scope.

10. Conclusion

D2.07 operationalizes D1.08's commitment to first-class conflict objects by specifying the detection mechanism that produces them, the record structure that carries them, the governed routing that assigns them to tiers, and the governance interpretation of aggregate detection results. Four properties define the compliant mechanism: detection integral to merge (not post-hoc); conflict records carrying both sides with attribution and class identification; routing governed by authored orchestration rules in the shared substrate rather than by hard-coded behavior; and conflict density readable as a governance compatibility signal rather than treated as a failure indicator. These properties inherit from Paper 1 Claim 2 and Paper 2's mating conflict registration, extended at inter-Self scope with the cross-perimeter attribution dimension that the joint-governance structure of FAI events requires.

Source Papers

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Inter-Self Coordination via Shared Substrate / Full Aspect Integration*. April 2026. ORCID: 0009-0004-8065-3235.

How to Cite This Note

Li, W. (2026). *FAI Conflict Detection at Merge*. May 15, 2026. ORCID: 0009-0004-8065-3235.